

CI/CD (Continuous integration and continuous deployment) - DevOps

I. Định nghĩa CI/CD

a. Định nghĩa, giải thích

- Continuous Integration (CI) là quá trình liên tục kiểm tra và tích hợp code mới vào code chính của một dự án.
- Giúp đưa mọi thay đổi được thực hiện bởi cá nhân lên một nơi chung để kiểm tra một cách tự động và sẽ đảm bảo rằng mọi thứ hoạt động một cách trơn tru hơn.
- Vd: khi commit code lên, tự động checker một CI pipeline: Build từ source code → thực hiện các bước kiểm thử hoặc phân tích tự động như Unit Test, Integration Test, E2E Test,... (tùy theo pipeline được cấu hình của từng dự án)

Những bước cơ bản

- Functional tests: kiểm tra phần mềm có hoạt động đúng theo mong đợi hay không, thường kiểm tra tự động mà đội ngũ QA (Quality Assurance) phát triển để đảm bảo tính năng hoạt động đúng
- Security scans: kiểm tra có lỗ hổng bảo mật nào không, vd bị SQL Injection hay XSS (Cross-Site Scripting)
- Code quality scan: đảm bảo code tuân thủ các tiêu chuẩn, vd: độ dài hàm, cách sử dụng khoảng trắng và các quy tắc khác
- Performance tests: kiểm tra xem code có đáp ứng được yêu cầu về hiệu suất không, chẳng hạn như thời gian xử lý yêu cầu hoặc khả năng chịu tải
- License scanning: kiểm tra tất cả các thư viện hoặc công cụ có sử dụng giấy phép phù hợp hay không, để tránh các vấn đề về pháp lý
- Fuzz testing: gửi các dữ liệu bất thường, vd chuỗi ký tự quá dài, số không hợp lệ, vào ứng dụng để kiểm tra xem nó có bị crash hay gặp lỗi bất ngờ không

b. CD là gì?

- Continuous
- Giúp tự động triển khai phiên bản đã vượt qua CI lên các môi trường như Development, Staging hoặc Production, tùy theo quy trình của dự án

Những bước cơ bản trong CD

- Build: biên dịch hoặc đóng gói ứng dụng thành artifact trước khi triển khai (nếu cần)
- Deploy: triển khai code: đẩy Docker image lên repository, dùng dòng lệnh để đưa code lên AWS, ... bằng cách tự động hoặc thủ công
- Artifact là sản phẩm sau quá trình Build như file JAR, WAR, Docker Image...

Lợi ích của CI/CD

- Giảm lỗi phát sinh: nhờ vào kiểm tra tự động → giúp phát hiện lỗi ngay từ sớm, tránh việc merge vào dev và gây lỗi toàn diện cho production → tốn chi phí và áp lực cho người làm web
- Tăng tốc độ phát triển: thay vì chờ đợi kiểm tra thủ công, mọi thứ được làm tự động hóa giúp việc kiểm tra và chạy sản phẩm nhanh chóng
- Tạo sự ổn định: do có kiểm tra và chạy tự động nên không lo triển khai nhầm phiên bản hoặc quên bước kiểm tra

II. CI/CD Pipeline:

a. Khái niệm:

- CI/CD Pipeline: chuỗi các bước được tự động hóa nhằm đưa mã nguồn từ khi lập trình viên commit đến khi triển khai lên môi trường thực tế. Pipeline giúp giảm thao tác thủ công, phát hiện lỗi sớm và đảm bảo quy trình phát hành phần mềm diễn ra nhất quán, ổn định

b. Quy trình tổng quát của 1 CI/CD Pipeline

Developer



Commit/ Push



Source Code Repository (GitHub, GitLab...)



CI Pipeline



Build



Kiểm thử



Code Quality / Security Scan



Đóng gói Artifact (JAR, Docker Image,...)



Artifact Repository



CD Pipeline



Deploy



Development / Staging / Production

c. Các thành phần chính của pipeline

- Source code repository: lưu trữ mã nguồn
- Build: biên dịch hoặc đóng gói ứng dụng
- Testing: thực hiện các kiểm thử tự động
- Static Code Analysis: phân tích chất lượng và bảo mật mã nguồn
- Artifact: tạo sản phẩm sau khi build
- Deploy: triển khai artifact lên môi trường tương ứng

III. Các công cụ cho CI/CD

a. GitHub Actions

1. Giới thiệu sơ lược

- Là công cụ CI/CD được tích hợp sẵn trong GitHub
- Cho phép tự động thực hiện các công việc như:
 - Build dự án
 - Chạy Unit Test
 - Kiểm tra chất lượng mã nguồn
 - Triển khai (Deploy) lên máy chủ hoặc dịch vụ Cloud
- Quy trình được định nghĩa bằng các file YAML đặt trong thư mục: ".github/workflows/"

2. Cách dùng

- Bước 1: Tạo repository
- Bước 2: Tạo thư mục: ".github/workflows/"
- Bước 3: Tạo file .yml trong thư mục workflows
- Bước 4: Khai báo thời điểm Workflow chạy để chạy tự động
- Bước 5: Chọn môi trường chạy để GitHub cung cấp một runner (máy ảo hoặc môi trường thực thi) để chạy workflow
- Bước 6: Checkout source code để GitHub Actions lấy source code mới nhất từ Repository
- Bước 7: Cài môi trường theo tùy dự án
- Bước 8: Build project
- Bước 9: Chạy tests
- Bước 10: Deploy (nếu có)

→ Sau mỗi lần Push thì GitHub sẽ tự động chạy workflow và hiển thị trạng thái, nếu lỗi thì sẽ hiển thị log để kiểm tra

b. Jenkins

1. Giới thiệu sơ lược

- Là công cụ mã nguồn mở hỗ trợ tự động hóa quy trình CI/CD
- Jenkins có thể tích hợp với GitHub, GitLab, Docker, Kubernetes, Maven,... thông qua hệ thống plugin phong phú
- Jenkins thường được cài đặt trên máy chủ riêng (self-hosted) và quản lý toàn bộ quá trình Build → Test → Deploy

2. Cách dùng

- Bước 1: Cài đặt Jenkins
 - Cài đặt Java do Jenkins được phát triển bằng Java
 - Cài đặt Jenkins và truy cập thông qua trình duyệt tại <http://localhost:8080>
 - Thực hiện cấu hình ban đầu và cài đặt các Plugin được đề xuất
 - Sử dụng GitHub Webhook kết hợp với một public domain (ví dụ thông qua ngrok) để Jenkins có thể nhận các sự kiện từ GitHub
- Bước 2: Cấu hình CI (Continuous Integration)
 - Cài đặt Plugin GitHub Pull Request Builder
 - Tạo Webhook trên GitHub Repository để Jenkins nhận sự kiện tạo Pull Request
 - Cấu hình thông tin Repository và tài khoản GitHub trên Jenkins
 - Tạo một CI Job, thiết lập điều kiện kích hoạt khi có Pull Request và xây dựng Pipeline thực hiện các bước như Build, Unit Test, Checkstyle và báo cáo kết quả
- Bước 3: Cấu hình CD (Continuous Delivery)
 - Cài đặt Plugin Generic Webhook Trigger
 - Tạo Webhook để nhận sự kiện khi Pull Request được hợp nhất (Merge)
 - Tạo một CD Job và xây dựng Pipeline gồm các giai đoạn như Checkout, Build, Deploy và Notify
 - Sau khi triển khai thành công, Jenkins gửi thông báo để người dùng biết quá trình triển khai đã hoàn tất

→ Jenkins tự động thực hiện Build, kiểm thử và triển khai khi nhận được sự kiện từ GitHub thông qua Webhook, giúp giảm thao tác thủ công và hỗ trợ quy trình CI/CD hiệu quả hơn

c. Argo CD

1. Giới thiệu sơ lược

- Công cụ GitOps hỗ trợ Continuous Deployment dành cho Kubernetes
- Theo mô hình GitOps, đồng bộ trạng thái triển khai từ Git Repository sang Kubernetes Cluster
- Thích hợp cho các hệ thống sử dụng container và Kubernetes

2. Cách dùng

- Bước 1: Chuẩn bị môi trường

- Triển khai Argo CD trên cụm Kubernetes
- Chuẩn bị Git Repository lưu trữ các tệp khai báo (manifest) của ứng dụng
- Kết nối Argo CD với Kubernetes Cluster và Git Repository để quản lý việc triển khai ứng dụng
- Bước 2: Lấy source manifest từ Git Repository
 - Repository Server thực hiện lấy (git pull) các manifest từ Git Repository
 - Lưu manifest vào bộ nhớ cục bộ (local cache)
 - Sinh ra các tệp YAML manifest để cung cấp cho Application Controller xử lý ở bước tiếp theo
- Bước 3: Đồng bộ với Kubernetes
 - Application Controller lấy trạng thái hiện tại của ứng dụng đang chạy trên Kubernetes
 - So sánh với trạng thái mong muốn được định nghĩa trong Git Repository
 - Nếu phát hiện khác biệt (OutOfSync), Argo CD tự động hoặc theo yêu cầu thực hiện Sync bằng cách áp dụng (apply) các manifest lên Kubernetes để đồng bộ hệ thống
- Bước 4: Hiển thị kết quả
 - API Server tiếp nhận thông tin từ các thành phần khác
 - Hiển thị trạng thái triển khai thông qua Web UI hoặc API
 - Cho phép người dùng theo dõi trạng thái, đăng nhập, phân quyền và thực hiện các thao tác như Sync hoặc Rollback khi cần

→ Argo CD tự động theo dõi Git Repository, phát hiện sự khác biệt giữa trạng thái mong muốn và trạng thái thực tế của Kubernetes, sau đó đồng bộ hệ thống và hiển thị kết quả để người dùng quản lý dễ dàng thông qua Web UI hoặc API

Nguồn tham khảo:

- Cho phần I: <https://www.youtube.com/watch?v=f9szfAZkVHY>
- Cho phần II: <https://viblo.asia/p/cicd-pipeline-la-gi-mot-cicd-pipeline-hoan-thien-trong-nhu-the-nao-3RL1BKl7Vao>
- Một số công cụ cho CI/CD: https://dev.to/dev_tips/50-best-cicd-tools-for-2025-the-ultimate-guide-to-automating-your-devops-pipeline-eh1
- Thông tin cho GitHub Actions: <https://fit.neu.edu.vn/post/ci-cd-voi-github-actions-tu-dong-hoa-kiem-thu-va-trien-khai-phan-mem>
- Thông tin cho Jenkins: <https://viblo.asia/p/tim-hieu-ve-jenkins-va-cicd-eW65GbDxIDO>
- Thông tin cho Argo CD: <https://viblo.asia/p/argo-cd-hieu-biet-co-ban-GAWVpOQaL05>